1: STUDENT REGISTRATION FORM USING HTML, CSS AND JAVASCRIPT

1. Problem Statement

Develop a responsive student registration form using HTML, CSS and JavaScript. The form should collect the student's name, roll number, email address, course, password and file upload. CSS should be used to improve the appearance and responsiveness of the form. JavaScript should validate required fields, check the email format and verify password strength before submission.

2. Technologies Used

HTML: Creates the registration form and its input fields.

CSS: Provides styling, layout and responsive design.

JavaScript: Validates user inputs and displays error messages.

3. Implementation

The registration form is implemented using three separate files:

1. index.html – Contains the form structure and input fields.
2. style.css – Defines the form layout, colours, spacing and responsive design.
3. script.js – Validates the input fields, email address and password strength.

The HTML file connects the CSS and JavaScript files to provide a complete registration interface.

4. Working of the Application

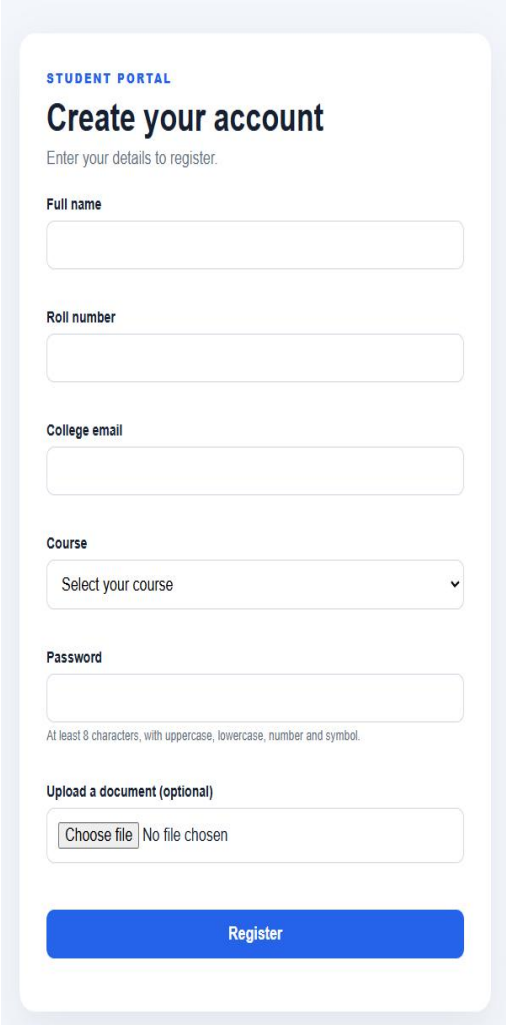
1. The user opens the student registration webpage.
2. The user enters their name, roll number, email address, course and password.
3. The user selects a file if required.
4. JavaScript checks whether the required fields are completed.

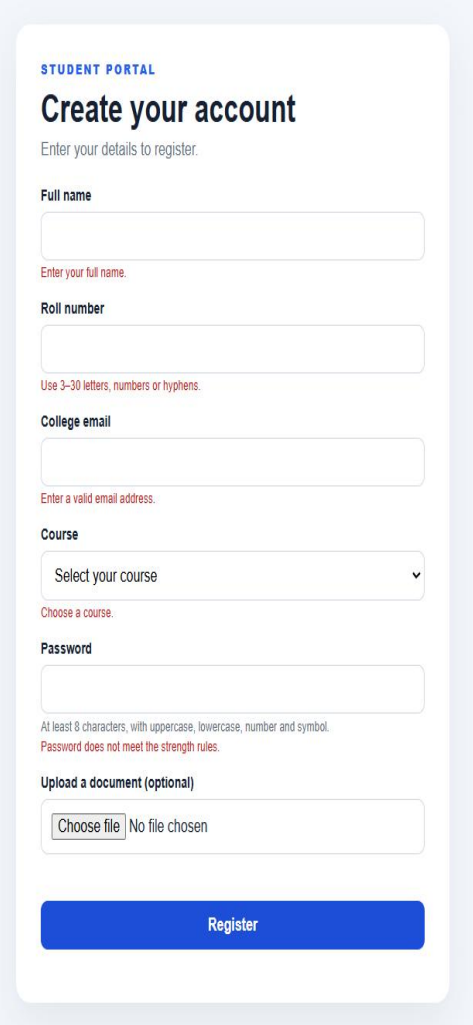
5.The email address is checked for a valid format.

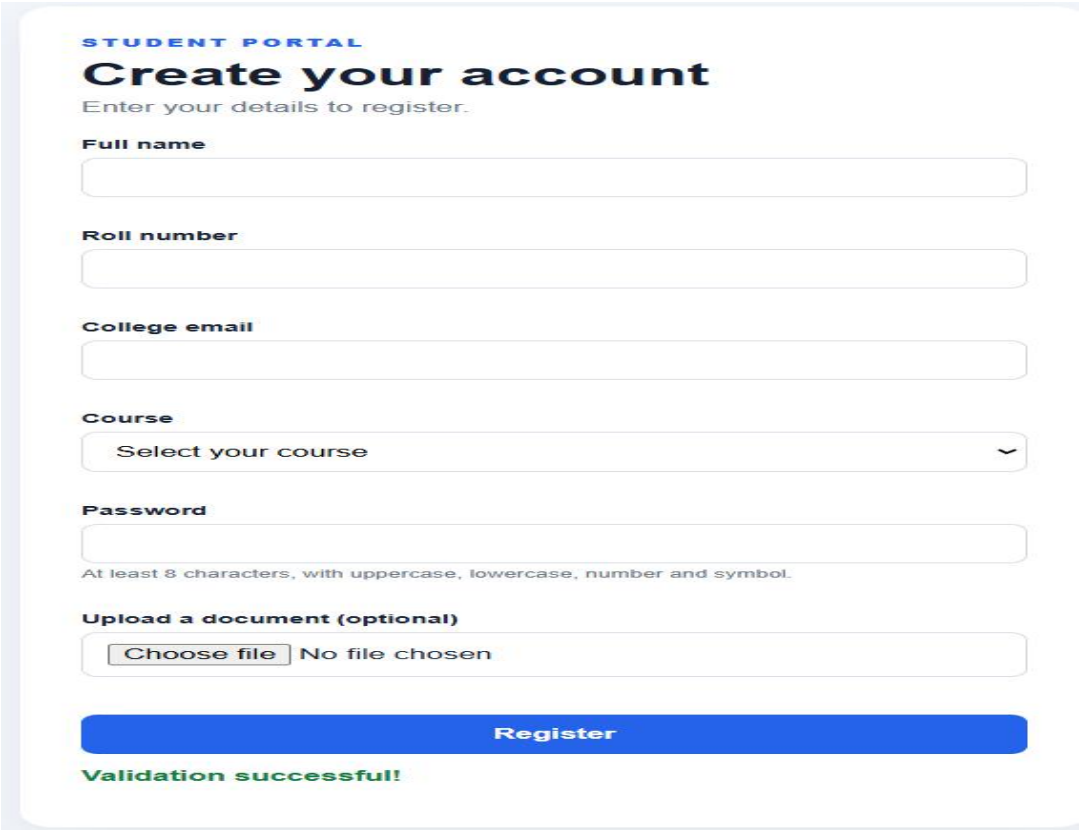
6.The password is checked for sufficient strength.

7.Appropriate validation messages are displayed when the entered details are invalid.

5. Output:







6. Code Explanation

INDEX.HTML

<!DOCTYPE html>

<html lang="en">

<head>

<!-- Defines character encoding and responsive page settings -->

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<!-- Sets the webpage title -->

<title>Student Registration</title>

```
    <!-- Connects the external CSS file -->
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <!-- Main container for the registration form -->
    <main class="card">
        <h1>Create Your Account</h1>
        <p>Enter your details to register.</p>

        <!-- Registration form with JavaScript validation -->
        <form id="registrationForm" novalidate>

            <!-- Collects the student's name -->
            <label for="name">Full Name</label>
            <input id="name" name="name" type="text" required>
            <small class="error" id="nameError"></small>

            <!-- Collects the student's roll number -->
            <label for="roll">Roll Number</label>
            <input id="roll" name="roll" type="text" required>
            <small class="error" id="rollError"></small>

            <!-- Collects the student's email address -->
            <label for="email">College Email</label>
```

```
<input id="email" name="email" type="email" required>
<small class="error" id="emailError"></small>

<!-- Allows the student to select a course -->
<label for="course">Course</label>
<select id="course" name="course" required>
   <option value="">Select your course</option>
   <option value="CSE">Computer Science</option>
   <option value="IT">Information Technology</option>
   <option value="ECE">Electronics and Communication</option>
   <option value="AIML">AI and Machine Learning</option>
</select>
<small class="error" id="courseError"></small>

<!-- Collects the student's password -->
<label for="password">Password</label>
<input id="password" name="password"
      type="password" required>
<small class="error" id="passwordError"></small>

<!-- Allows an optional document upload -->
<label for="document">Upload Document (Optional)</label>
<input id="document" name="document" type="file"
      accept=".pdf,.jpg,.jpeg,.png">
<small class="error" id="documentError"></small>
```

```
            <!-- Submits the form for validation -->
            <button type="submit">Register</button>

            <!-- Displays the validation success message -->
            <p id="successMessage" role="status"></p>
        </form>
    </main>
     <!-- Connects the external JavaScript file -->
    <script src="script.js"></script>
</body>
</html>
```

STYLE.CSS

```
/* Applies consistent sizing to all elements */
* {
    box-sizing: border-box;
}

/* Sets the default font and page colours */
:root {
    font-family: Arial, sans-serif;
    color: #172033;
    background: #f3f6fb;
}

/* Centres the registration form on the page */
body {
    margin: 0;
    min-height: 100vh;
    display: grid;
    place-items: center;
    padding: 24px;
}

/* Styles the main registration card */
.card {
    width: min(100%, 560px);
```

```
    background: white;
    padding: 32px;
    border-radius: 18px;
    box-shadow: 0 16px 45px #17203314;
}

/* Styles the main heading */
h1 {
    font-size: 30px;
}

/* Displays labels above the input fields */
label {
    display: block;
    font-weight: bold;
    margin: 15px 0 7px;
}

/* Styles input fields and dropdown menus */
input, select {
    width: 100%;
    padding: 12px;
    border: 1px solid #d6dce7;
    border-radius: 9px;
    font: inherit;
}

/* Highlights the selected input field */
input:focus, select:focus {
    outline: 2px solid #2563eb;
}

/* Styles the Register button */
button {
    width: 100%;
    padding: 13px;
    margin-top: 22px;
    background: #2563eb;
    color: white;
    border: none;
    border-radius: 9px;
    cursor: pointer;
}

/* Changes the button colour when hovered */
button:hover {
    background: #1d4ed8;
}

/* Displays validation errors in red */
```

```
.error {
   display: block;
   color: #c62828;
   font-size: 12px;
}

/* Displays success messages in green */
#successMessage {
   color: green;
   font-weight: bold;
}

/* Adjusts the form for smaller screens */
@media (max-width: 480px) {
   body {
      padding: 12px;
   }

   .card {
      padding: 22px;
   }
}
```

SCRIPT.JS

```
// Selects the registration form

const form = document.getElementById("registrationForm");



// Displays an error message beside the specified field

function setError(id, message) {

   document.getElementById(id + "Error").textContent = message;

}



// Runs validation when the user submits the form

form.addEventListener("submit", (event) => {



   // Prevents normal form submission
```

```
event.preventDefault();

// Assumes the form is valid initially
let valid = true;

// Clears previous error messages
document.querySelectorAll(".error").forEach(item => {
    item.textContent = "";
});

// Clears the previous success message
document.getElementById("successMessage").textContent = "";

// Retrieves the values entered by the user
const name = document.getElementById("name").value.trim();
const roll = document.getElementById("roll").value.trim();
const email = document.getElementById("email").value.trim();
const course = document.getElementById("course").value;
const password = document.getElementById("password").value;

// Retrieves the selected file, if available
const file = document.getElementById("document").files[0];

// Checks whether the name contains at least two characters
if (name.length < 2) {
```

```
    setError("name", "Enter your full name.");
    valid = false;
}

// Checks the roll number format
if (!/^[A-Za-z0-9-]{3,30}$/.test(roll)) {
    setError("roll", "Enter a valid roll number.");
    valid = false;
}

// Checks the email address format
if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
    setError("email", "Enter a valid email address.");
    valid = false;
}

// Checks whether a course has been selected
if (!course) {
    setError("course", "Choose a course.");
    valid = false;
}

// Checks password length and character requirements
if (!/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^A-Za-z0-9]).{8,}$/.test(password)) {
    setError("password", "Use a strong password.");
```

```
        valid = false;
    }
    // Rejects files larger than 2 MB
    if (file && file.size > 2 * 1024 * 1024) {
        setError("document", "File must be 2 MB or smaller.");
        valid = false;
    }
    // Allows only PDF, JPEG and PNG files
    if (file && !["application/pdf", "image/jpeg", "image/png"].includes(file.type)) {
        setError("document", "Upload only PDF, JPG or PNG.");
        valid = false;
    }
    // Displays success only when every validation passes
    if (valid) {
        document.getElementById("successMessage").textContent =
            "Validation successful! Registration is not saved.";

        // Resets the input fields after successful validation
        form.reset();
    }
});
```

7. Outcome

The student registration interface provides a responsive form with client-side validation using HTML, CSS and JavaScript.

STUDENT MANAGEMENT SYSTEM USING PHP AND MYSQL

1. Problem Statement

Develop a student management system using PHP and MySQL to perform CRUD operations on student details. The system should allow users to add, view, edit and delete student records. It should also support uploading profile images or documents, storing file metadata in the database, displaying uploaded files and validating file size and type to prevent invalid uploads.

2. Technologies Used

HTML: Creates the user interface.

CSS: Styles the web pages.

PHP: Processes student details and uploaded files.

MySQL: Stores student information and file metadata.

XAMPP: Provides the local Apache server and MySQL database.

3. Database Design

Create a database named student_db with two tables.

Table 1: students

Field	Description
id	Unique student ID
name	Student name
roll_no	Student roll number
email	Student email address
course	Student course

Table 2: student_files

Field	Description
id	Unique file ID
student_id	Related student ID
file_name	Original file name
file_path	Stored file path
file_type	Uploaded file type
upload_date	File upload date

The student_id field links uploaded files to their corresponding student records.

4. Database Creation — schema.sql

```
-- Creates the database
CREATE DATABASE IF NOT EXISTS student_db;

-- Selects the database
USE student_db;

-- Creates the student details table
CREATE TABLE IF NOT EXISTS students (
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   roll_no VARCHAR(30) NOT NULL UNIQUE,
   email VARCHAR(100) NOT NULL,
   course VARCHAR(100) NOT NULL
);

-- Creates the uploaded file metadata table
CREATE TABLE IF NOT EXISTS student_files (
   id INT AUTO_INCREMENT PRIMARY KEY,
   student_id INT NOT NULL,
   file_name VARCHAR(255) NOT NULL,
   file_path VARCHAR(255) NOT NULL,
   file_type VARCHAR(100),
   upload_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

   -- Connects each file to its student
   FOREIGN KEY (student_id) REFERENCES students(id)
   ON DELETE CASCADE
);
```

5.PHP Implementation

The application contains separate PHP pages for adding, viewing, editing and deleting student records.

A. Database Connection — db.php

```
<?php
// Connects PHP to the MySQL database
$conn = new mysqli("localhost", "root", "", "student_db");

// Checks whether the database connection failed
if ($conn->connect_error) {
   die("Database connection failed");
}

// Sets the character encoding
$conn->set_charset("utf8mb4");
?>
```

B. Add Student and Upload File — add.php

```
<?php

// Includes the database connection

require "db.php";

$message = "";

// Processes the form when submitted

if ($_SERVER["REQUEST_METHOD"] === "POST") {

   // Collects and validates student details

   $name = trim($_POST["name"] ?? "");

   $roll = trim($_POST["roll_no"] ?? "");

   $email = trim($_POST["email"] ?? "");

   $course = trim($_POST["course"] ?? "");

   if ($name === "" || $roll === "" ||

       !filter_var($email, FILTER_VALIDATE_EMAIL) ||

       $course === "") {
```

```
    $message = "Enter valid student details.";
} else {
    // Uses a prepared statement to insert student details
    $stmt = $conn->prepare(
        "INSERT INTO students (name, roll_no, email, course)
         VALUES (?, ?, ?, ?)"
    );
    $stmt->bind_param("ssss", $name, $roll, $email, $course);
    if ($stmt->execute()) {
        // Gets the ID of the newly added student
        $studentId = $conn->insert_id;
        // Processes a file only when one was selected
        if (isset($_FILES["document"]) &&
            $_FILES["document"]["error"] === UPLOAD_ERR_OK) {
           $file = $_FILES["document"];
            // Checks the maximum file size of 2 MB
            if ($file["size"] > 2 * 1024 * 1024) {
                $message = "Student added, but file is too large.";
            } else {
                // Determines the actual MIME type of the file
                $finfo = new finfo(FILEINFO_MIME_TYPE);
                $type = $finfo->file($file["tmp_name"]);
                // Allows only PDF, JPEG and PNG files
                $allowed = [
```

```
            "application/pdf" => "pdf",
            "image/jpeg" => "jpg",
            "image/png" => "png"
        ];
        if (isset($allowed[$type])) {

            // Creates the upload folder if necessary
            if (!is_dir("uploads")) {
                mkdir("uploads", 0755, true);
            }
            // Generates a unique filename to prevent overwriting
            $fileName = bin2hex(random_bytes(16))
                      . "." . $allowed[$type];
            $path = "uploads/" . $fileName;
            // Moves the uploaded file into the folder
            if (move_uploaded_file(
                $file["tmp_name"], $path
            )) {
                // Stores file metadata in the database
                $fs = $conn->prepare(
                    "INSERT INTO student_files
                    (student_id, file_name, file_path, file_type)
                    VALUES (?, ?, ?, ?)"
                );
$originalName = basename($file["name"]);
```

```
                $fs->bind_param(
                    "isss", $studentId,
                    $originalName, $path, $type
                );
                if ($fs->execute()) {
                    $message = "Student and file added successfully.";
                } else {
                    unlink($path);
                    $message = "File metadata could not be saved.";
                }
                $fs->close();
            } else {
                $message = "File upload failed.";
            }
        } else {
            $message = "Only PDF, JPG and PNG files are allowed.";
        }
    }
} else {
    $message = "Student added successfully.";
}
} else {
    $message = "Unable to add student. Check for duplicate roll numbers.";
}
$stmt->close();
```

```
    }
}
?>
<!-- Displays the add-student form -->
<h2>Add Student</h2>
<p><?= htmlspecialchars($message) ?></p>
<form method="post" enctype="multipart/form-data">
    <!-- Collects student details -->
    Name: <input name="name" required><br>
    Roll Number: <input name="roll_no" required><br>
    Email: <input name="email" type="email" required><br>
    Course: <input name="course" required><br>
    <!-- Allows optional file upload -->
    Document:
    <input type="file" name="document"
        accept=".pdf,.jpg,.jpeg,.png"><br>
    <button type="submit">Add Student</button>
</form>
```

6.View Student Records — view.php

```
<?php
// Connects to the database
require "db.php";
// Retrieves student records
$result = $conn->query("SELECT * FROM students ORDER BY id DESC");
```

```
?>
<h2>Student Records</h2>
<table border="1" cellpadding="8">
    <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Roll Number</th>
        <th>Email</th>
        <th>Course</th>
        <th>Files</th>
        <th>Actions</th>
    </tr>
    <?php while ($row = $result->fetch_assoc()): ?>
    <tr>
        <!-- Displays escaped student information -->
        <td><?= (int)$row["id"] ?></td>
        <td><?= htmlspecialchars($row["name"]) ?></td>
        <td><?= htmlspecialchars($row["roll_no"]) ?></td>
        <td><?= htmlspecialchars($row["email"]) ?></td>
        <td><?= htmlspecialchars($row["course"]) ?></td>
        <td>
            <?php
            // Retrieves files linked to this student
            $stmt = $conn->prepare(
                "SELECT file_name, file_path
```

```
            FROM student_files WHERE student_id = ?"
        );
        $stmt->bind_param("i", $row["id"]);
        $stmt->execute();
        $files = $stmt->get_result();
        // Displays every uploaded file
        while ($file = $files->fetch_assoc()):
        ?>
            <a href="<?= htmlspecialchars($file["file_path"]) ?>">
                <?= htmlspecialchars($file["file_name"]) ?>
            </a><br>
        <?php endwhile; $stmt->close(); ?>
    </td>
    <!-- Provides edit and delete links -->
    <td>
        <a href="edit.php?id=<?= (int)$row["id"] ?>">Edit</a>
        <a href="delete.php?id=<?= (int)$row["id"] ?>">Delete</a>
    </td>
  </tr>
  <?php endwhile; ?>
</table>
```

7. Edit Student — edit.php

```
<?php
// Connects to the database
```

```
require "db.php";
// Retrieves the student ID from the URL
$id = filter_input(INPUT_GET, "id", FILTER_VALIDATE_INT);
if (!$id) {
    exit("Invalid student ID.");
}
// Updates student details when the form is submitted
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    // Collects updated details
    $name = trim($_POST["name"] ?? "");
    $email = trim($_POST["email"] ?? "");
    $course = trim($_POST["course"] ?? "");
    // Validates the input
    if ($name === "" || $course === "" ||
        !filter_var($email, FILTER_VALIDATE_EMAIL)) {
        exit("Enter valid details.");
    }
    // Updates the selected student record
    $stmt = $conn->prepare(
        "UPDATE students SET name=?, email=?, course=? WHERE id=?"
    );
    $stmt->bind_param("sssi", $name, $email, $course, $id);
    $stmt->execute();
    $stmt->close();
    // Redirects to the student list
```

```
    header("Location: view.php");
    exit;
}
// Retrieves the current student details
$stmt = $conn->prepare("SELECT * FROM students WHERE id=?");
$stmt->bind_param("i", $id);
$stmt->execute();
$student = $stmt->get_result()->fetch_assoc();
$stmt->close();
if (!$student) {
    exit("Student not found.");
}
?>
<!-- Displays the current details in the edit form -->
<h2>Edit Student</h2>
<form method="post">
    Name:
    <input name="name"
        value="<?= htmlspecialchars($student["name"]) ?>" required><br>
    Email:
    <input name="email" type="email"
        value="<?= htmlspecialchars($student["email"]) ?>" required><br>
    Course:
    <input name="course"
        value="<?= htmlspecialchars($student["course"]) ?>" required><br>
```

```
    <button type="submit">Update Student</button>
</form>
```

7. Delete Student — delete.php

```
<?php
// Connects to the database
require "db.php";
// Accepts deletion through POST only
if ($_SERVER["REQUEST_METHOD"] !== "POST") {
    exit("Invalid request.");
}
// Retrieves and validates the student ID
$id = filter_input(INPUT_POST, "id", FILTER_VALIDATE_INT);
if (!$id) {
    exit("Invalid student ID.");
}
// Deletes the selected student using a prepared statement
$stmt = $conn->prepare("DELETE FROM students WHERE id=?");
$stmt->bind_param("i", $id);
$stmt->execute();
// Related file metadata is removed by ON DELETE CASCADE
$stmt->close();
// Returns to the student list
header("Location: view.php");
exit;
```

```
?>
```

For example, replace the Delete link in view.php with:

```
<!-- Sends the delete request using POST -->
<form method="post" action="delete.php"
      onsubmit="return confirm('Delete this student?');">
    <input type="hidden" name="id"
           value="<?= (int)$row['id'] ?>">
    <button type="submit">Delete</button>
</form>
```

8.Output

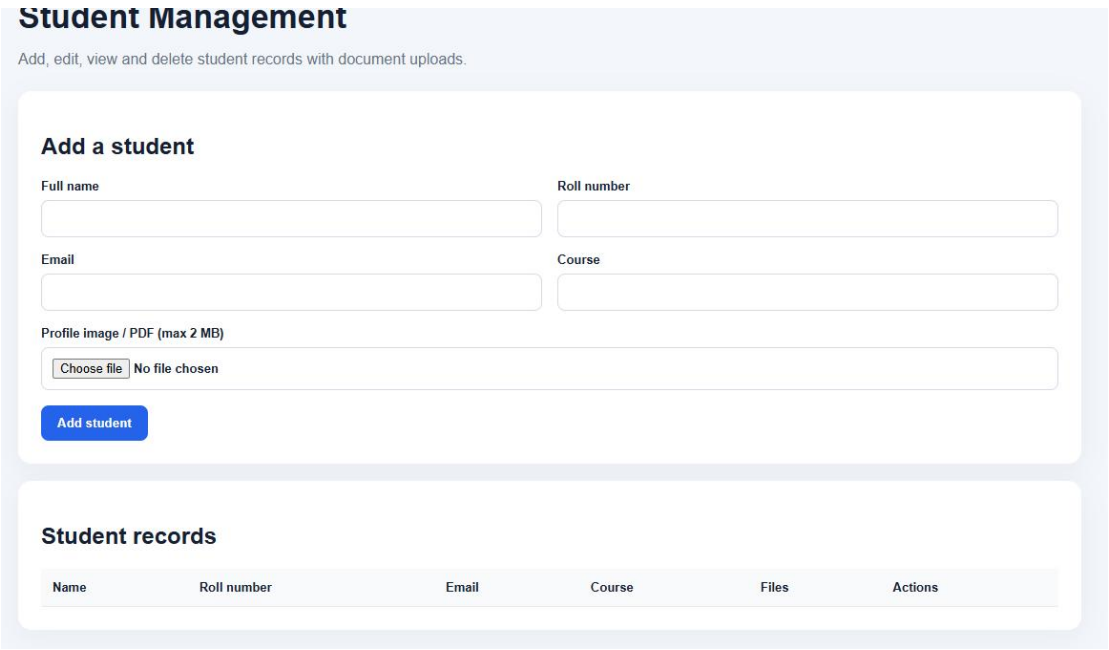


FIGURE 2.1 Students Record

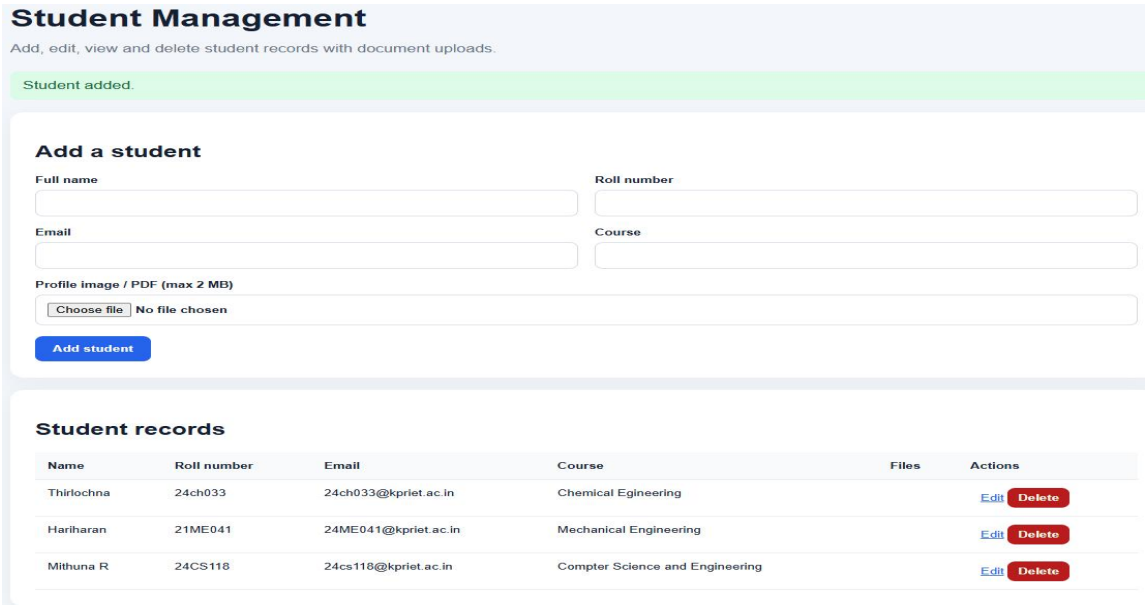


Figure 2.2— Add Student

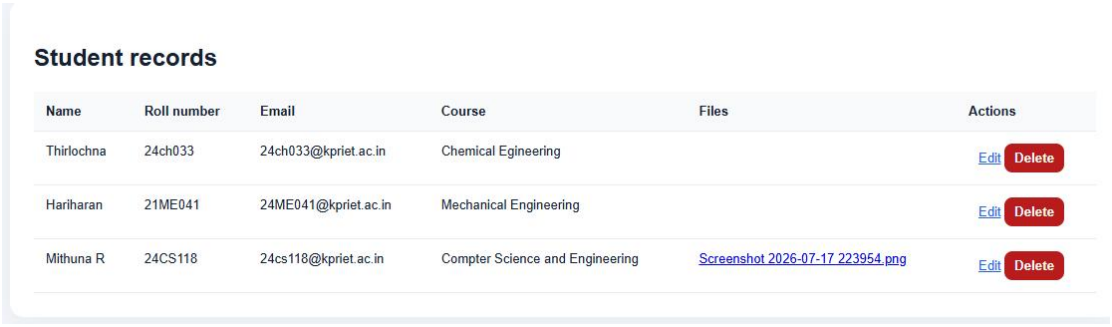


Figure 2.3 View record

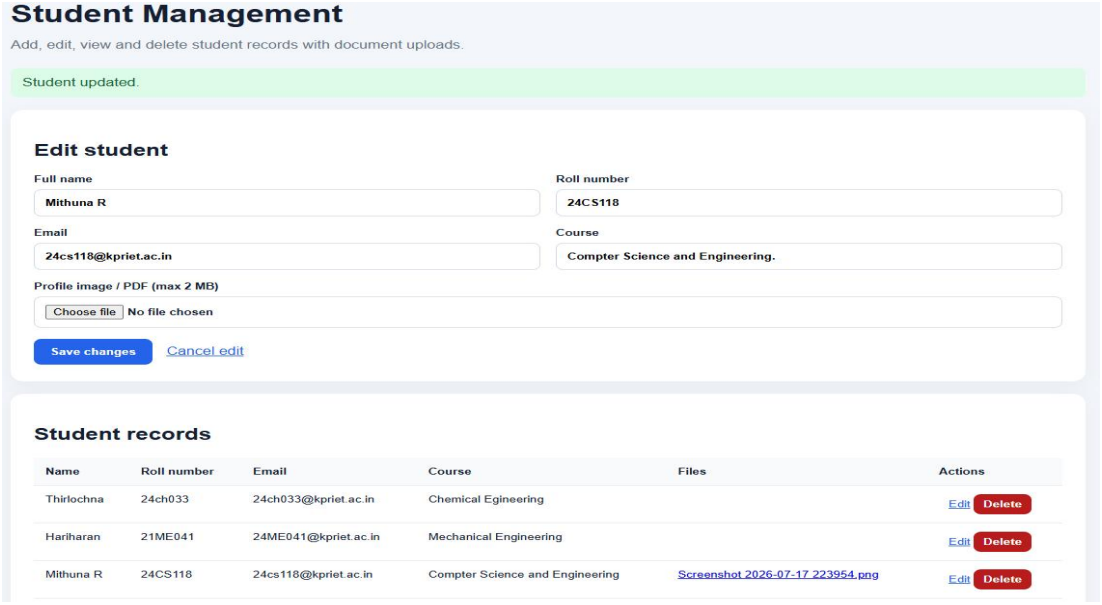


Figure 2.4 Student Update



Figure 2.5 File Uploaded



Figure 2.6 Student Deleted.

9. Outcome

The Student Management System was developed using PHP and MySQL to perform CRUD operations on student records. The application supports adding, viewing, editing and deleting student details. It also allows users to upload profile images and PDF documents, validates file types and sizes, and stores file metadata in the database. Thus, the application demonstrates student record management and file handling through a web-based interface.

STUDENT PORTAL USING JAVA SERVLETS AND JDBC

1. Problem Statement

Develop a web-based student portal using Java Servlets, JDBC and MySQL. The application should retrieve student details and associated file paths from the database. A Servlet should stream images and documents from a storage folder. Student profiles should be displayed dynamically using HTML or JSP, with images loaded through Servlet URLs. The application should support multiple student records and files and use session handling to restrict access to authenticated users.

2. Technologies Used

- **Java:** Implements server-side application logic.
- **Java Servlets:** Handles HTTP requests and responses.
- **JDBC:** Connects the Java application to MySQL.
- **MySQL:** Stores student details and file metadata.
- **JSP/HTML:** Displays student profiles dynamically.
- **Apache Tomcat:** Runs the Java web application.

3. System Requirements

Hardware Requirements

- Computer or laptop
- Minimum 4 GB RAM
- Internet connection for downloading required software

Software Requirements

- Windows operating system
- Java Development Kit (JDK)
- Apache Tomcat Server
- MySQL Server
- Visual Studio Code
- Apache Maven

4. System Design

The Student Portal follows a client-server architecture.

1. The user opens the Student Portal using a web browser.
2. The login page accepts the username and password.
3. The Login Servlet validates the demo credentials.
4. A session is created after successful authentication.
5. The Student Servlet retrieves student records from the MySQL database using JDBC.
6. JSP displays the retrieved information dynamically.
7. Images and documents can be served from the server through a Servlet.

5. Implementation

5.1 Login Module

The login module authenticates users before allowing access to student records. It receives the username and password from the login form and checks them against the configured demo credentials.

If the credentials are valid, a session is created and the user is redirected to the student details page. Otherwise, an error message is displayed on the login page.

Sample Code:

```
private static final String DEMO_USER = "student";
private static final String DEMO_PASSWORD = "ChangeMe123!";


if (DEMO_USER.equals(username) &&
    DEMO_PASSWORD.equals(password)) {


    HttpSession session = request.getSession(true);
    session.setAttribute("loggedIn", Boolean.TRUE);
    session.setMaxInactiveInterval(20 * 60);


    response.sendRedirect(
        request.getContextPath() + "/students"
    );
}
```

5.2 Database Connectivity Using JDBC

JDBC is used to connect the Java application to the MySQL database. The database stores student information and file paths required by the application.

Sample Code:

```
Connection con = DriverManager.getConnection(

"jdbc:mysql://localhost:3306/webtech_assignment?useSSL=false&serverTimezone=
UTC",
   "root",
   ""
);
```

5.3 Retrieving Student Records

The Student Servlet retrieves student information from the MySQL database and forwards the records to the JSP page for display.

Sample Code:

```
PreparedStatement ps =
   con.prepareStatement("SELECT * FROM students");

ResultSet rs = ps.executeQuery();

while (rs.next()) {
   String name = rs.getString("name");
   String email = rs.getString("email");
}
```

5.4 Dynamic Student Profile Display

JSP is used to display student information retrieved from the database. The page can display details such as student name, email, and available file links.

Loops are used to process multiple student records. Conditional statements can be used to display messages when records or files are unavailable.

Images can be displayed using the HTML <img> element. When image streaming is implemented through a Servlet, the image URL points to that Servlet.

Sample HTML:

```
<div class="student-card">
   <h3>Student Details</h3>
   <p>Name: Student Name</p>
   <p>Email: student@example.com</p>
   <img src="image?id=1" alt="Student Profile">
</div>
```

5.5 File Handling

The application is designed to retrieve student images and documents stored on the server using their file paths.

A Servlet can read a file and stream its contents to the browser. This allows supported images and documents to be displayed or opened when requested.

5.6 Session Management

Session management restricts access to student records to authenticated users.

6. Database Design

Database Name: webtech_assignment

Table: students

The student table stores student information required by the portal. Depending on the implemented schema, it may contain fields such as:

Field	Description
id	Unique student identifier
name	Student name
email	Student email address
image_path	Path to the student's profile image
document_path	Path to an associated document

8. Output Screenshots

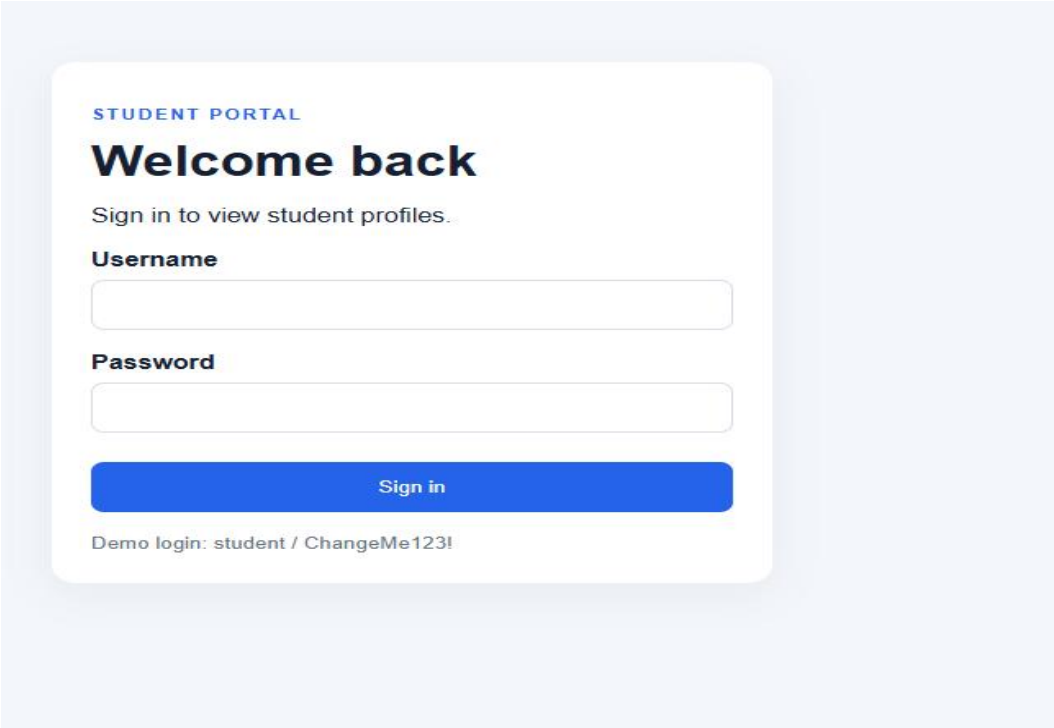


Figure 3.1: Student Portal Login Page

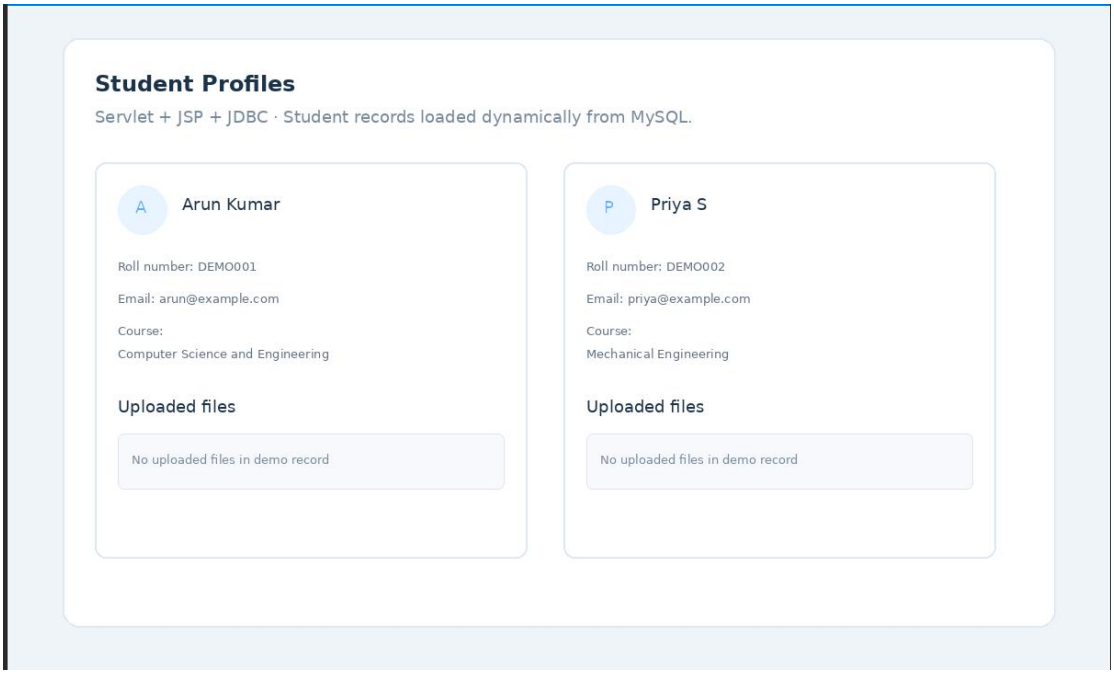


Figure 2: Student Details Page

9. Conclusion

The Student Portal application is designed using Java Servlets, JSP, JDBC, and MySQL. It demonstrates user authentication, session management, database connectivity, and the dynamic display of student information. The application also supports serving student images and documents from server storage through Servlet-based endpoints. These features demonstrate how Java web technologies can be integrated to build a database-driven student information system.

The database connection and JDBC driver configuration must be corrected and tested before the complete application can be considered successfully executed.